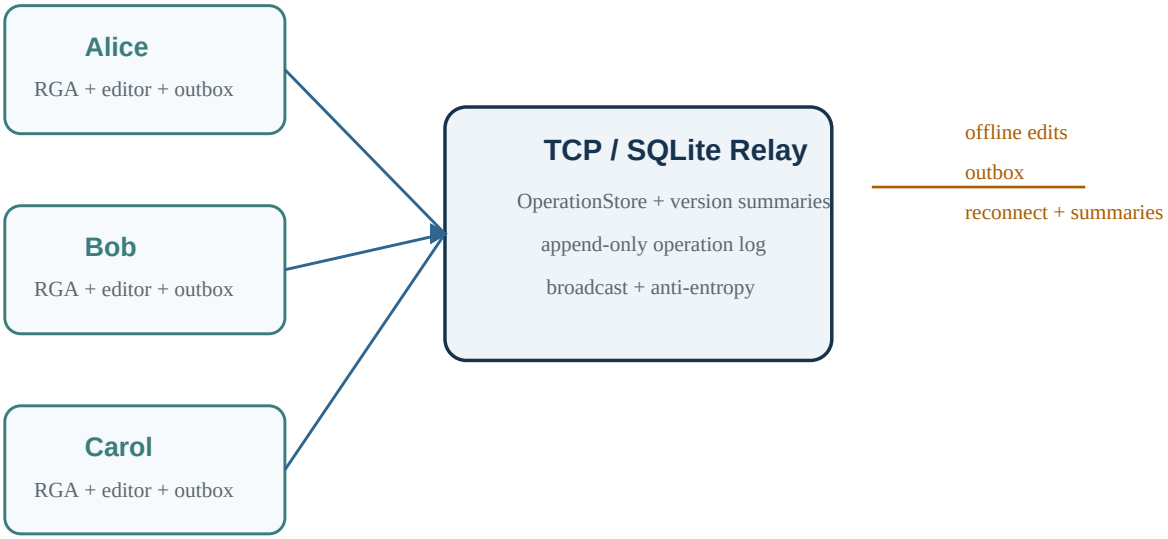
25. Real-Time Collaborative Text Editor

A verified sequence-CRDT implementation with offline editing, anti-entropy synchronization, and persistent TCP relay



Report basis	Current project source files and recorded verification output
Implementation	Python RGA sequence CRDT; in-memory and TCP/SQLite relay paths
Verification status	24 unit/integration tests passed; independent-process E2E passed
Generated	16 October 2026

Submission-quality technical report

Contents

- 1. Project overview
- 2. Problem statement and objectives
- 3. Assignment requirements and implementation mapping
- 4. System architecture and component responsibilities
- 5. RGA sequence CRDT design
- 6. Identifiers, insertion, deletion, and tombstones
- 7. Deterministic ordering and convergence
- 8. Offline editing, reconnect, and anti-entropy
- 9. TCP relay and SQLite persistence
- 10. Adversarial delivery and idempotency
- 11. Tombstone compaction and membership safety
- 12. Demo results and metrics
- 13. Testing methodology and actual results
- 14. File-by-file implementation summary
- 15. Limitations and future improvements
- 16. Conclusion

Evidence policy

All technical claims in this report are derived from the current project files inspected before generation and the verification outputs recorded during the current session. The report distinguishes the in-memory demonstration relay from the persistent TCP/SQLite relay and does not claim production readiness.

1. Project overview

This project implements a real-time collaborative text editor around a Replicated Growable Array (RGA) sequence CRDT. Text is represented as a deterministic ordered structure of character nodes. Each operation is uniquely identified by a replica name and counter, carries causal dependency information and a Lamport timestamp, and can be delivered more than once or out of order.

Two relay paths are present. The in-memory relay supports deterministic simulations and the demonstration program. The TCP relay uses newline-delimited JSON and SQLite persistence for real client processes. Clients can disconnect, continue editing locally, reconnect, upload operations missing from the relay, and request operations missing locally.

The verified workflow includes exact offline insertion of **OFFLINE** before **Hello!**, convergence of three independent processes, concurrent insertion/deletion, shuffled and duplicated delivery, restart-safe counters, legacy operation decoding, malformed-input tolerance, and conservative tombstone compaction.

2. Problem statement and objectives

The system must provide collaborative text editing without a central character order being required for every edit. Replicas may create operations concurrently, lose connectivity temporarily, receive messages late, or receive duplicates. The resulting document must converge when all replicas eventually receive the same operations.

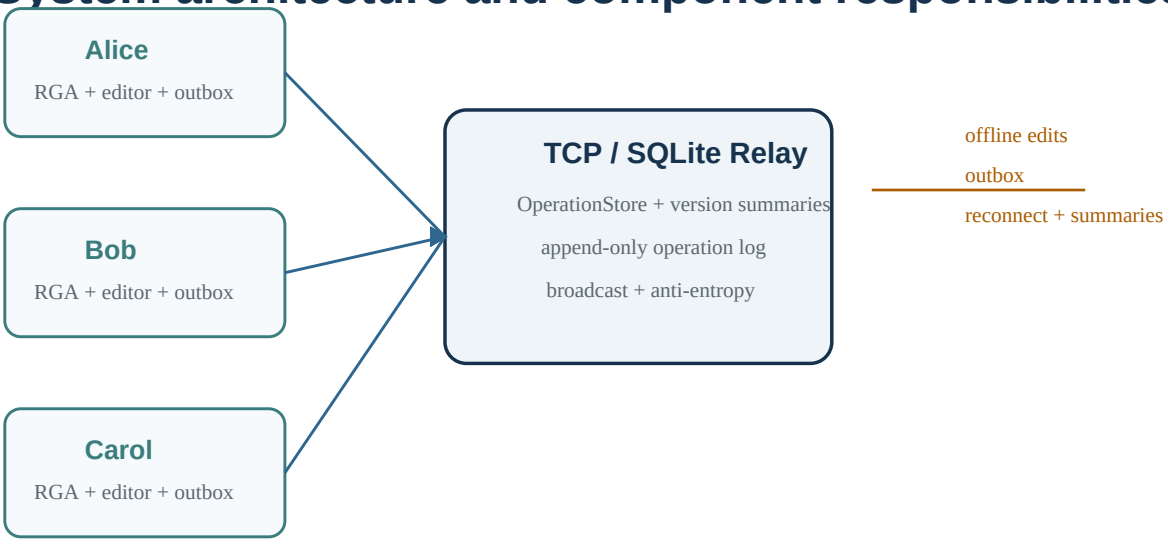
- Represent character insertions and deletions as replicated operations.
- Preserve user-visible text order for multi-character insertion.
- Make operation application idempotent and independent of delivery order.
- Support local editing while disconnected and synchronize after reconnect.
- Persist operations in a relay so a restarted relay can recover history.
- Measure convergence, metadata growth, relay-log bytes, and forwarding activity.
- Define safe tombstone removal without invalidating anchors or pending dependencies.

3. Assignment requirements and implementation mapping

Requirement	Implementation evidence	Status
Sequence CRDT with unique IDs	RGA nodes use (replica, counter) identifiers.	PASS
Insertions and deletion tombstones	Operation.kind is insert/delete; deleted nodes remain structurally present.	PASS
Minimal editor	editor.py and CLI insert/delete/show/disconnect/reconnect commands.	PASS
Persistent three-client relay	network_relay.py ThreadingTCPServer plus SQLite OperationStore.	PASS
Offline/reconnect anti-entropy	NetworkClient outbox, relay summaries, reconnect upload and missing-op delivery.	PASS
Duplicate idempotency	RGA.operations and pending IDs reject repeats; relay suppresses duplicate storage/broadcast.	PASS
Shuffled/delayed delivery	shuffled_delivery and randomized convergence tests.	PASS
Concurrent insertion/deletion	CRDT test and demo report result 'A'.	PASS
Hash convergence	Three-process test compares exact SHA-256 hashes.	PASS
Metrics	synchronization_report and demo output.	PASS

Requirement	Implementation evidence	Status
Safe tombstone removal	Stable minimum summaries, anchor/pending checks, membership omission rejection.	PASS

4. System architecture and component responsibilities



Component	Responsibility
crdt_editor.py	Operation model, RGA state, Lamport ordering, pending operations, tombstones, summaries, compaction, in-memory relay/client, metrics.
editor.py	Small editor façade with cursor movement, insertion, backspace, and rendering.
network_client.py	TCP client lifecycle, reader thread, locking, outbox, offline commands, reconnect anti-entropy, machine-readable READY output.
network_relay.py	Threaded TCP server, SQLite operation persistence, summaries, missing-operation delivery, duplicate suppression, peer broadcast, malformed JSON tolerance.
demo.py	In-memory demonstration of concurrent edits, convergence, compaction, and metrics.
test_*.py	Unit, socket integration, persistence, and independent-process verification.

5. RGA sequence CRDT design

An RGA is represented as a rooted ordered graph. The root is the string sentinel `HEAD`. Every insert creates one character node and records its anchor. The visible document is obtained by a pre-order traversal of the graph, excluding nodes whose tombstone flag is true.

The implementation keeps an indexed parent-to-children map rather than scanning every node for children on each traversal. Traversal is iterative, using an explicit stack, so document length does not consume Python recursion depth.

An operation is immutable and serializable. Its fields are:

- `op_id`: replica and counter, also the node ID for inserts.
- `kind`: insert or delete.
- `anchor/value`: insert location and one character.
- `target`: deletion target.

- `deps`: contiguous per-replica causal summary.
- `lamport`: deterministic logical timestamp.

6. Identifiers, insertion, deletion, and tombstones

Local counters are advanced when operations are created. When a replica applies an operation bearing its own replica ID, it also advances its counter to the maximum observed counter. This prevents a restarted replica from generating an identifier already present in replayed history.

Multi-character input is converted into a causal chain. The first character anchors at the requested position; each following character anchors to the preceding newly created character. Thus `OFFLINE` remains in input order even when inserted at index zero, in the middle, or at the end.

```
operations = crdt.make_inserts(index, text)
for operation in operations:
    client.publish(operation)
```

Deletion does not remove a node immediately. It marks the node as deleted and records a delete operation. The node remains available as an anchor for descendants and as a target for dependency resolution until safe compaction is proven.

7. Deterministic ordering and convergence

Each replica advances its Lamport clock before creating a local operation and updates the clock when applying any received operation. Siblings sharing an anchor are sorted by descending `(lamport, replica, counter)`. This is a total deterministic key; it does not use a potentially cyclic mixture of dependency comparisons.

Causal chains preserve a later insertion's position relative to the operation it depends on. Concurrent siblings use the remaining replica/counter tie-breakers. Operations that reference an unseen anchor or delete target enter a pending map and are drained once the dependency becomes available.

The tests exercise 100 shuffled Lamport-order histories and 25 randomized three-replica histories with insertions and deletions. Duplicate delivery is also explicitly applied twice. All participating replicas are required to produce the same document hash.

8. Offline editing, reconnect, and anti-entropy

A client disconnects only its network connection. Its RGA state and local editor remain active. Locally generated operations are applied immediately and placed in an outbox. Reconnect performs a handshake, obtains the relay's summary, uploads local operations whose counters exceed the relay's summary, uploads queued operations, and requests operations missing from the client.

The network client protects CRDT access with an RLock. Each reader thread owns its file object, and an old reader cannot mark a newer connection offline. Socket shutdown precedes close so blocked readers are unblocked. Failed sends are caught and the operation is returned to the outbox.

```
disconnect
insert 0 OFFLINE
show
reconnect
show
```

The independent-process test performs this workflow with Alice, Bob, and Carol as separate processes using explicit READY acknowledgements and bounded timeouts rather than manual input.

9. TCP relay and SQLite persistence

network_relay.py implements a threaded TCP server using newline-delimited JSON. Each client identifies a room and replica. The relay stores operation payloads in an SQLite table keyed by room, replica, and counter. Duplicate inserts return false and are not rebroadcast.

A ready response includes stored-byte information and the relay's contiguous per-replica summary. The relay can send missing operations to a reconnecting client and broadcasts newly stored operations to connected peers. Malformed JSON lines are ignored so a bad line does not terminate the handler.

The relay log is append-only in this project. Local CRDT tombstone compaction does not rewrite or delete relay history; a production snapshot/compaction protocol would need a stable vector and coordinated membership.

10. Adversarial delivery and idempotency

RGA.apply first checks whether an operation ID is already known or pending. This makes duplicates no-ops. Operations with missing anchors or delete targets are retained in the pending map and applied after their dependencies arrive. The in-memory shuffled_delivery helper duplicates the operation list and randomizes delivery order.

The relay independently suppresses duplicate database rows and only broadcasts operations newly accepted by OperationStore.put. This separates local CRDT idempotency from relay persistence idempotency.

11. Tombstone compaction and membership safety

The current safe_compact API is deliberately fail-closed about membership. It requires a mapping of labeled summaries and an explicit set of relevant replica IDs. If any relevant ID is absent, it raises ValueError instead of treating the omitted replica as if it had acknowledged nothing or as if it did not exist.

```
safe_compact(
    replica_summaries: Mapping[str, Dict[str, int]],
    relevant_replicas: Iterable[str],
)
```

After membership validation, the CRDT computes the component-wise minimum summary. A tombstone is removable only when its delete operation is covered by that minimum, no child remains anchored to the node, and no pending insertion or deletion references it. The delete-target index avoids an $O(n^2)$ scan.

Authoritative membership limitation: the CRDT cannot discover a replica that the caller never declares relevant. The application or relay must provide the authoritative replica-membership set. A permanently offline replica must remain in that set; its unacknowledged summary blocks removal. This is a safety contract, not a claim that the local CRDT has global membership knowledge.

12. Demo results and metrics

The demo uses the in-memory Relay, not the TCP/SQLite server. The recorded output from the current implementation was:

```
concurrent insertion/deletion result: 'A'
converged: True
metadata before delete: [44, 44, 44]
metadata after delete: [85, 85, 85]
metadata after stable compaction: [42, 42, 42]
sync bytes: {'relay_log_bytes': 101, 'relay_forwarded_bytes': 202}
```

Metric	Meaning
Document hashes	SHA-256 of each visible document string.
metadata_bytes	Serialized estimate of CRDT node metadata; not Python process memory.
relay_log_bytes	Serialized operation bytes in the in-memory relay log.
relay_forwarded_bytes	In-memory forwarding accounting for serialized operation payloads; not complete TCP wire traffic.

13. Testing methodology and actual results

Verification was performed against the current code. Existing databases and running user processes were not reset or stopped. Process-level tests use isolated temporary SQLite databases.

Command	Actual result
python -m unittest -v	Ran 24 tests in 7.131s — OK
python -m unittest -v test_process_e2e.py	1 test in 1.288s — OK
python -m py_compile (recursive *.py)	Passed with no errors
python demo.py	Passed; converged=True; hashes identical

The independent-process E2E test verifies exact text `OFFLINEHello!` and identical 64-character hexadecimal document hashes for Alice, Bob, and Carol. The full suite includes exact beginning/middle/end insertion tests, randomized convergence, restart counters, malformed JSON, legacy Lamport compatibility, relay persistence, offline outbox behavior, and compaction safety.

14. File-by-file implementation summary

File	Role
crdt_editor.py	Core Operation, Version, RGA, summaries, Lamport ordering, pending operations, tombstones, compaction, in-memory relay/client, metrics.
editor.py	Cursor-oriented editor façade.
network_client.py	Thread-safe TCP client, offline lifecycle, outbox, anti-entropy, CLI and machine mode.
network_relay.py	Threaded persistent relay and SQLite OperationStore.
demo.py	In-memory concurrent-edit and metrics demonstration.
test_crdt_editor.py	CRDT correctness, randomized delivery, insertion order, restart, compaction tests.
test_network_e2e.py	Socket, persistence, malformed-input, offline/reconnect integration tests.
test_process_e2e.py	Independent relay/Alice/Bob/Carol process-level exact-content test.
README.md	Run instructions, architecture, metrics, compaction contract, limitations.

15. Known limitations and future improvements

- No authentication or TLS is implemented.
- There is no explicit message-size limit.
- Independent clients must use distinct replica IDs; duplicate identity coordination is not implemented.
- The relay operation log is append-only and does not shrink through CRDT compaction.
- Forwarded-byte metrics are not complete TCP wire-level accounting.
- Tombstone safety depends on an authoritative membership set supplied by the application or relay.
- A production system should add membership/lease management, snapshots, stable-vector coordination, authentication, TLS, rate limiting, and operational monitoring.

16. Conclusion

The current project provides an end-to-end collaborative editor prototype that meets the assignment's functional goals: sequence-CRDT editing, deterministic convergence, deletion tombstones, offline operation, reconnect anti-entropy, persistent relay storage, duplicate tolerance, concurrent-edit demonstration, metrics, and automated process-level verification.

The evidence is bounded and honest. The implementation is not presented as production-ready: security, resource limits, replica identity ownership, and coordinated relay snapshotting remain future work. Within the tested scope, the recorded suite and independent-process workflow passed, and the current code's compaction API explicitly prevents accidental omission of a declared relevant replica.

End of report